MemryLab: Tracking Personal Belief Evolution Through Multi-Source Digital Footprint Analysis

TUSHAR LAAD, Independent Researcher

People today generate an unprecedented volume of personal text across dozens of digital platforms—journals, social media posts, messaging conversations, notes, and data exports—yet no existing tool helps them understand how their own thinking evolves over time. Current personal information management systems excel at cataloging and retrieving past events, but they fail to surface the meta-narrative: how beliefs shift, when contradictions emerge, and what arcs of personal growth look like across years of heterogeneous data.

We present MemryLab, a privacy-preserving desktop application that fuses personal data exports from 30+ platforms into a unified temporal knowledge base and applies an eight-stage LLM-powered analysis pipeline to extract themes, beliefs, sentiments, entities, contradictions, and evolution narratives. The system introduces a novel contradiction detection algorithm that combines embedding-based semantic pre-filtering with LLM verification to identify belief shifts with sub-quadratic complexity. A hybrid retrieval-augmented generation (RAG) pipeline, combining BM25 full-text search with cosine-similarity vector retrieval via Reciprocal Rank Fusion, enables conversational exploration of personal history augmented by long-term memory facts.

All processing runs locally on the user's machine through a provider-agnostic LLM integration layer supporting nine inference backends, from local Ollama models to cloud APIs. The system is implemented as a 4.7 MB Tauri 2.0 desktop application with a hexagonal Rust backend and React frontend, storing all data in a single encrypted SQLite database. We evaluate the system on import throughput, analysis quality, contradiction detection accuracy, and search relevance, and present a longitudinal case study demonstrating the discovery of belief evolution patterns invisible to the participant prior to analysis.

CCS Concepts: • **Human-centered computing → Interactive systems and tools**; • **Information systems → Information extraction**.

Additional Key Words and Phrases: personal information management, belief evolution, digital footprint analysis, contradiction detection, retrieval-augmented generation, local-first AI

1 Introduction

The average internet user maintains accounts on more than eight social media platforms [35] and generates thousands of messages, posts, notes, and journal entries each year. Under regulations such as the EU General Data Protection Regulation (GDPR) [34] and the California Consumer Privacy Act (CCPA), individuals can now download comprehensive archives of their digital activity from virtually every major platform. Yet despite this unprecedented access to one's own data, no widely available tool exists to help people understand the longitudinal story their digital footprint tells about their personal evolution.

Existing personal information management (PIM) systems [5, 6] are designed for storage and retrieval: they help users find a specific email, locate a saved bookmark, or search through notes. Digital memory systems such as MyLifeBits [2] and more recently Microsoft Recall [4] capture and index life experiences for later retrieval. Journaling applications like Day One offer tagging and search within a single platform. None of these systems address what we call the *meta-narrative problem*: the challenge of understanding not just *what* one wrote, but *how* one's thinking has changed over time.

Consider a person who, in 2019, wrote in their journal "I value stability above all else" and who, by 2023, was posting on social media about embracing uncertainty and taking career risks. This shift in fundamental belief is invisible to any

Author's Contact Information: Tushar Laad, Independent Researcher, tusharlaad2002@gmail.com.

tool that treats documents as independent retrieval targets. Surfacing such patterns requires (1) ingesting heterogeneous data from multiple platforms, (2) extracting structured beliefs and preferences from unstructured text, (3) tracking those beliefs temporally, and (4) detecting when they contradict or evolve.

The emergence of large language models (LLMs) creates a new opportunity. Unlike traditional NLP pipelines that require task-specific training data, modern LLMs can extract nuanced beliefs, classify sentiments, and generate narrative syntheses in a zero-shot or few-shot manner [31, 32]. However, applying LLMs to deeply personal data raises acute privacy concerns: users are understandably reluctant to send their journals, therapy notes, and private messages to cloud-based APIs.

We present MemryLab, a system that addresses these challenges through four contributions:

(1) **Multi-source data fusion with confidence-scored auto-detection.** A trait-based adapter architecture that parses data exports from 30+ platforms (social media, messaging, notes, journals, streaming services) with automatic format detection via confidence scoring, two-pass import strategy, and SHA-256 content deduplication.
(2) **Temporal belief extraction and contradiction detection pipeline.** An eight-stage analysis pipeline that extracts themes, sentiments, beliefs, and entities from personal text, then applies a novel contradiction detection algorithm combining embedding-based semantic pre-filtering with LLM verification to identify belief shifts with sub-quadratic complexity.
(3) **Evolution arc detection with LLM-powered narrative synthesis.** A temporal differencing system that compares theme snapshots across configurable time windows and generates second-person prose narratives describing the user's personal evolution with supporting evidence.
(4) **Privacy-preserving, provider-agnostic architecture.** A local-first design where all data remains on the user's device in an encrypted SQLite database, with a hexagonal architecture that decouples the analysis pipeline from any specific LLM provider, supporting nine backends from fully-local Ollama inference to optional cloud APIs.

The remainder of this paper is organized as follows. Section 2 surveys related work across five relevant areas. Section 3 describes the system architecture including the data fusion and storage layers. Section 4 details the eight-stage analysis pipeline. Section 5 covers the query and retrieval system. Section 6 discusses the privacy architecture. Section 7 summarizes implementation details. Section 8 presents quantitative evaluation and a longitudinal case study. Section 9 discusses implications and limitations, and Section 10 concludes.

2 Related Work

2.1 Personal Information Management

The vision of a personal memory augmentation system dates to Vannevar Bush's Memex [1], a hypothetical device that would allow individuals to store, annotate, and traverse their accumulated knowledge through associative trails. Eight decades later, this vision remains largely unrealized.

Gordon Bell's MyLifeBits project [2, 3] was the most ambitious attempt to capture a complete digital life, ingesting emails, documents, photographs, and web pages into a searchable database. While MyLifeBits demonstrated the feasibility of comprehensive life capture, it focused on storage and retrieval rather than analysis of how the individual's thinking evolved.

More recently, Microsoft's Recall system [4] captures periodic screenshots of a user's activity and applies OCR and semantic indexing to enable "photographic memory" retrieval. Recall addresses the recall problem (finding past information) but not the reflection problem (understanding how one has changed).

The PIM research community has long recognized that people organize personal information in idiosyncratic ways [5–7]. Boardman and Sasse [8] found that cross-tool fragmentation is a primary barrier to personal information coherence—people maintain separate organizational schemes for email, files, and bookmarks with little integration. MemryLab addresses this fragmentation directly by unifying 30+ data sources into a single temporal knowledge base.

2.2 Knowledge Graphs and Entity Extraction

Personal knowledge graphs (PKGs) have been proposed as a means of organizing an individual's information landscape [9, 10]. Unlike enterprise knowledge graphs, PKGs must handle deeply heterogeneous sources, evolve continuously, and respect privacy constraints.

Named entity recognition (NER) from heterogeneous documents has been extensively studied [11, 12], but most systems assume homogeneous input formats. The challenge in personal data is that entity mentions span informal social media posts, structured journal entries, and conversational messages—each with different conventions for referring to people, places, and organizations.

Temporal knowledge graphs [13, 14] extend static knowledge graphs with time annotations, enabling reasoning about how facts change. Our entity extraction pipeline produces temporally-grounded entity mentions that can be aggregated into a personal temporal knowledge graph with relationship tracking via recursive common table expressions (CTEs).

2.3 Memory-Augmented AI Systems

A recent wave of research has focused on giving AI agents persistent memory capabilities, motivated by the observation that conversational AI systems lack the ability to accumulate and reason over long-term knowledge.

Mem0 [15] introduces a self-improving memory layer for AI agents that extracts, stores, and retrieves facts from conversations. Mem0 distinguishes between short-term (session) memory and long-term (persistent) memory, using an LLM to decide what facts to retain. MemryLab's memory store draws inspiration from this design, implementing a `MemoryFact` model with confidence scores, categories (Belief, Preference, Fact, SelfDescription, Insight), and temporal metadata.

MemoRAG [16] proposes a memory-inspired approach to retrieval-augmented generation that uses a lightweight model to form a "global memory" of the corpus, improving retrieval for ambiguous or multi-hop queries. While MemoRAG focuses on improving RAG quality, its insight that global corpus understanding improves retrieval directly motivates our memory augmentation in the RAG pipeline.

NEMORI [17] introduces a self-organizing memory system for autonomous agents with structured memory hierarchies and consolidation processes inspired by human episodic and semantic memory. This biological inspiration parallels our distinction between raw document storage (episodic) and extracted beliefs and preferences (semantic).

The Hindsight framework [18] addresses the problem of opinion formation in AI agents, using networks of opinion-holding agents to model how beliefs evolve through interaction. While Hindsight focuses on multi-agent systems, its formalization of opinion evolution informs our single-user belief tracking model.

Nair et al. [19] propose episodic memory mechanisms for RAG systems, demonstrating that memory of past retrieval interactions improves future retrieval quality. Our system incorporates a related idea: persistent memory facts derived from analysis are injected into RAG context to improve answer quality for personal queries.

2.4 Digital Footprint Analysis

Research on digital footprint analysis has demonstrated that personal digital traces contain rich psychological signals. Kosinski et al. [21] showed that Facebook Likes alone can predict personality traits, political orientation, and other sensitive attributes with remarkable accuracy. Youyou et al. [22] extended this work, finding that computer-based personality judgments from digital footprints are more accurate than those made by humans.

Sentiment analysis over time has been applied to social media streams to detect mood patterns [23, 24], life events [25], and mental health indicators [26]. However, these approaches typically analyze a single platform in isolation and focus on aggregate sentiment rather than specific belief evolution.

A related challenge arises in LLM-based knowledge systems: knowledge conflicts [20] occur when different sources provide contradictory information about the same topic. Xu et al. [20] survey this problem in the context of LLMs, identifying conflicts between parametric knowledge, retrieved context, and user input. Our contradiction detection system applies a related formulation to the intra-personal setting: detecting when a user's own beliefs, expressed at different times and on different platforms, conflict with each other.

2.5 Privacy-Preserving AI

The local-first software movement [27] advocates for applications that keep data on the user's device, synchronize peer-to-peer, and function without cloud dependencies. This philosophy aligns with privacy-preserving AI, where the goal is to derive insights from personal data without exposing it to third parties.

Federated learning [28] enables model training across distributed devices without centralizing data, but requires coordination infrastructure that is impractical for single-user desktop applications. On-device inference has become feasible through projects such as Ollama[1], llama.cpp [29], and the broader ecosystem of quantized open-weight models [30].

MemryLab adopts a pragmatic approach: all data storage and processing runs locally by default, with optional cloud LLM usage that is transparently logged so the user knows exactly what data has been sent externally. This differs from systems that claim privacy through policy ("we won't look at your data") by providing privacy through architecture (the data never leaves the device unless the user explicitly configures a cloud provider).

3 System Architecture

3.1 Design Principles

MemryLab's architecture is guided by four principles:

Privacy by default. The system operates as a zero-knowledge application: no data leaves the user's device unless a cloud LLM provider is explicitly configured, and all such transmissions are logged. The single-file SQLite database can be encrypted via SQLCipher, and API keys are stored in the operating system's secure keychain (Windows Credential Manager, macOS Keychain, or Linux Secret Service).

Hexagonal architecture. The system follows the ports-and-adapters pattern [33], defining abstract *port* interfaces for all external dependencies (LLM inference, embedding generation, document storage, vector search) and providing concrete *adapter* implementations. This decouples business logic from infrastructure, enabling the analysis pipeline to run identically whether backed by a local Ollama instance or a cloud API.

[1] https://ollama.com

Provider-agnostic LLM integration. Rather than coupling to a single model or API, the system defines `ILlmProvider` and `IEmbeddingProvider` port traits with four core operations: `complete`, `classify`, `is_available`, and `provider_name`. Nine adapter implementations cover Ollama, OpenAI, Anthropic Claude, Google Gemini, Groq, Mistral, Together AI, Fireworks AI, and any OpenAI-compatible endpoint. A usage logging decorator transparently records all LLM calls with token counts and latencies.

Two-pass exploratory import. Data ingestion uses a two-pass strategy: the first pass runs all platform-specific adapters with confidence scoring to identify the best parser for each file or archive, and the second pass applies a generic adapter to catch any remaining content. This design favors recall over precision in data ingestion, ensuring no personal content is silently dropped.

3.2 Multi-Source Data Fusion

The data fusion subsystem is built around the `SourceAdapter` trait, which every platform adapter implements:

Listing 1. The SourceAdapter trait (Rust).

```
pub trait SourceAdapter: Send + Sync {
    fn metadata(&self) -> SourceAdapterMeta;
    fn detect(&self, file_listing: &[&str])
        -> f32; // confidence 0.0-1.0
    fn parse(&self, path: &Path)
        -> Result<Vec<Document>, AppError>;
    fn name(&self) -> &str;
}
```

The `metadata()` method returns rich self-registration information including display name, icon, data export URL, user-facing instructions, accepted file extensions, and whether the adapter handles ZIP archives. The `detect()` method receives a listing of file paths within an import target and returns a confidence score between 0.0 and 1.0. The registry evaluates all adapters and selects the one with the highest confidence above a threshold of 0.3.

Table 1 summarizes the 33 platform adapters with their detection signatures and supported export formats.

Deduplication. Each document is assigned a SHA-256 hash of its content at parse time. Before insertion, the deduplication stage queries the document store by content hash; documents with an existing hash are silently skipped. This enables idempotent re-import: users can import the same data export multiple times without creating duplicates, which is critical for incremental import workflows where a user re-downloads their data export periodically.

Normalization and chunking. After deduplication, documents pass through a normalizer that standardizes Unicode encoding, strips excessive whitespace, and resolves platform-specific markup. A chunker then splits documents into overlapping segments of configurable size (default: 512 tokens with 50-token overlap) for downstream embedding and analysis.

3.3 Storage Architecture

All data resides in a single SQLite database operating in WAL (Write-Ahead Logging) mode for concurrent read access during import operations. The storage layer comprises seven specialized stores, each implemented as a port interface with a SQLite adapter:

Table 1. Supported platform adapters with detection signatures. Confidence thresholds represent the minimum score returned by the `detect()` method for a positive match.

Category	Platform	Detection Signature	Export Format
Journals & Notes	Day One	`*.doentry`, JSON manifest	JSON entries + photos
	Obsidian	`.obsidian/` directory	Markdown vault
	Notion	Notion export structure	Markdown/CSV
Social Media	Twitter/X	`data/tweets.js`	JS data archive
	Facebook	`your_activity_across_facebook/`	JSON/HTML
	Instagram	`content/posts_1.json`	JSON
	Reddit	`comments.csv`, `posts.csv`	CSV
	LinkedIn	`Connections.csv`	CSV
Messaging	WhatsApp	`_chat.txt` pattern	Text export
	Telegram	`result.json`	JSON
	Discord	`messages/` directory	JSON per channel
	Slack	Slack export structure	JSON
	Signal	Signal backup format	Encrypted backup
Fediverse	Mastodon	`outbox.json` ActivityPub	JSON-LD
	Bluesky	`bsky-` prefix files	AT Protocol
	Threads	Threads export structure	JSON
Content	Substack	`posts/` with HTML	HTML
	Medium	Medium export structure	HTML
	Tumblr	Tumblr export structure	HTML/JSON
Media & Services	YouTube	`watch-history.json`	JSON (Takeout)
	Spotify	`StreamingHistory` files	JSON
	Netflix	`ViewingActivity.csv`	CSV
	TikTok	TikTok data structure	JSON
Productivity	Google Takeout	`Takeout/` root directory	Mixed formats
	Evernote	`*.enex` files	ENEX (XML)
	Apple Notes	Apple export structure	Various
	Microsoft	Microsoft data export	Various
Commerce	Amazon	Amazon order/review data	CSV/JSON
	Pinterest	Pin export structure	JSON
Generic	Markdown	`*.md` files	Markdown
	Snapchat	Snapchat Memories export	JSON

Document Store. Stores raw documents and their chunks with full metadata including source platform, timestamp, participants, and content hash. Supports retrieval by ID, content hash, platform, and time range.

Page Index (FTS5). A full-text search index built on SQLite's FTS5 extension [39] with BM25 ranking [36]. The index covers chunk text and document metadata, enabling sub-millisecond keyword search across the entire corpus.

Vector Store. Stores embedding vectors for document chunks and supports approximate nearest-neighbor search via cosine similarity:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}||||\mathbf{b}||} \tag{1}$$

Embeddings are generated by the configured embedding provider (default: `nomic-embed-text` [38] via Ollama, or cloud alternatives).

Graph Store. An entity-relationship graph stored in relational tables with recursive CTE queries for relationship traversal. Nodes represent entities (people, places, organizations, concepts) and edges represent co-occurrence, mention, and semantic relationships. Graph traversal queries use SQLite's `WITH RECURSIVE` syntax to find multi-hop paths between entities.

Memory Store. A Mem0-inspired [15] long-term fact store that holds `MemoryFact` records with fields for fact text, source chunk references, confidence score, category (Belief, Preference, Fact, SelfDescription, Insight), temporal metadata (first seen, last updated), contradiction references, and active/inactive status. The memory store supports recall (fuzzy text search for relevant facts), storage, updating, and contradiction marking.

Timeline Store. A temporal index that maps documents to time ranges and supports queries like "documents in Q3 2022" or "monthly document counts." Provides the temporal scaffolding for the analysis pipeline's time-windowed processing.

Config Store. Stores application settings, LLM provider configuration, prompt versions, and analysis run history.

3.4 LLM Integration Layer

The LLM integration layer defines two port interfaces:

Listing 2. LLM port interfaces (Rust).

```
#[async_trait]
pub trait ILlmProvider: Send + Sync {
    async fn complete(&self, prompt: &str,
        params: &LlmParams) -> Result<String>;
    async fn classify(&self, text: &str,
        categories: &[String],
        params: &LlmParams) -> Result<Classification>;
    async fn is_available(&self) -> bool;
    fn provider_name(&self) -> &str;
}

#[async_trait]
pub trait IEmbeddingProvider: Send + Sync {
    async fn embed(&self, text: &str)
        -> Result<Vec<f32>>;
    async fn embed_batch(&self, texts: &[String])
        -> Result<Vec<Vec<f32>>>;
    fn dimensions(&self) -> usize;
}
```

Nine adapter implementations are provided: Ollama (local), OpenAI, Anthropic Claude, Google Gemini, Groq, Mistral, Together AI, Fireworks AI, and a generic OpenAI-compatible adapter that supports any endpoint implementing the OpenAI chat completions API.

A `UsageLogger` decorator wraps any `ILlmProvider` and transparently records every call with provider name, model identifier, prompt token count, completion token count, and wall-clock latency. This enables the privacy-critical feature of showing users exactly what data was sent to cloud providers and at what cost.

Eight-stage analysis pipeline. Documents flow through theme extraction, sentiment tracking, belief extraction, entity extraction, insight generation, contradiction detection, evolution arc detection, and narrative generation.

Fig. 1. Overview of the eight-stage analysis pipeline. Stages 1–5 extract structured information from documents; stages 6–8 synthesize higher-order patterns from extracted information.

4 Analysis Pipeline

The analysis pipeline transforms raw personal documents into structured insights through eight sequential stages (Figure 1), orchestrated by a central `run_analysis` function that coordinates all stages and aggregates results.

4.1 Eight-Stage Pipeline

4.1.1 Stage 1: Theme Extraction. The theme extraction stage partitions the document corpus into time windows (configurable as daily, weekly, monthly, quarterly, or yearly) and identifies dominant themes within each window. For each time window, the system samples up to 30 representative chunks, concatenates them into a context block, and prompts the LLM to identify recurring topics with intensity scores.

The output is a `ThemeSnapshot` model containing the time window boundaries, a list of theme labels with intensity scores (0.0–1.0), and supporting evidence in the form of chunk references. Theme labels are normalized across windows to enable temporal comparison: the LLM is provided with themes from the previous window and asked to reuse labels where the same theme persists.

4.1.2 Stage 2: Sentiment Tracking. Each document chunk is classified on a 5-point sentiment scale (very negative, negative, neutral, positive, very positive) using the LLM's `classify` method. To manage API costs, sentiment classification is applied to a stratified sample of up to 20 chunks per analysis run, with sampling weighted toward more recent content.

Sentiment scores are stored with their chunk IDs and timestamps, enabling temporal aggregation at any granularity. The frontend can visualize sentiment trends as time series, revealing patterns such as seasonal mood variations or sentiment shifts correlated with life events.

4.1.3 Stage 3: Belief Extraction. The belief extraction stage identifies explicit and implicit beliefs, preferences, and self-descriptions in the user's text. For each sampled chunk, the LLM is prompted with a structured extraction template that requests:

- **Beliefs**: Statements about how the world works ("I think remote work is more productive")
- **Preferences**: Expressions of taste or priority ("I prefer deep work in the morning")
- **Self-descriptions**: Statements about identity ("I'm an introvert who recharges alone")

Each extracted fact is stored as a `MemoryFact` with a confidence score derived from the LLM's self-assessed certainty, the specificity of the source text, and whether the fact is stated explicitly or inferred. Facts include bidirectional references to their source chunks for provenance tracking.

4.1.4 Stage 4: Entity Extraction. The entity extraction stage identifies named entities—people, places, organizations, and concepts—from document chunks using LLM-based extraction with structured output parsing. Up to 15 chunks are processed per analysis run, with the LLM returning entities in a JSON format that specifies entity name, type, and contextual description.

Extracted entities are inserted into the graph store as nodes, with edges created between entities that co-occur within the same chunk. Temporal metadata (the timestamp of the source document) enables queries such as "who was I talking about in 2021?" and visualization of how social networks evolve.

4.1.5 Stage 5: Insight Generation. The insight generation stage synthesizes patterns from the themes and beliefs extracted in earlier stages. The LLM receives the full set of current themes (with intensity scores) and beliefs (with confidence scores) and is prompted to identify non-obvious patterns, connections between themes, and implications of belief clusters.

Insights are stored as `MemoryFact` records with the `Insight` category and a default confidence of 0.7. Each insight includes references to the supporting themes and beliefs that motivated it, enabling the user to trace any insight back to its evidential basis.

4.1.6 Stage 6: Contradiction Detection. Contradiction detection is the novel analytical contribution of MemryLab. The challenge is to identify pairs of beliefs or preferences that are semantically contradictory, among a potentially large set of extracted facts. Naïve pairwise comparison has $O(n^2)$ complexity, which is prohibitive for large fact stores when each comparison requires an LLM call.

Our algorithm uses a two-phase approach (Algorithm 1):

Phase 1: Semantic pre-filtering. All active Belief and Preference facts (those with more than five words, to exclude trivially short entries) have their embeddings computed. Pairwise cosine similarity is computed, and only pairs exceeding a threshold $\tau = 0.7$ are retained as candidates. This filters out the vast majority of pairs: most beliefs are about unrelated topics and have low embedding similarity. The embedding phase is $O(n)$ in LLM calls (one per fact), and the pairwise similarity computation is $O(n^2)$ but involves only fast vector dot products, not LLM inference.

Phase 2: LLM verification. For each candidate pair (up to $k = 20$ pairs per run to bound cost), both facts are sent to the LLM with a structured prompt requesting a JSON response with three fields: `is_contradiction` (boolean), `explanation` (string), and `severity` (low/medium/high).

Confirmed contradictions are stored with bidirectional references: if fact A contradicts fact B, both A's `contradicted_by` list and B's list are updated. This enables the frontend to display contradiction networks and allows the RAG pipeline to surface contradictions when relevant.

The total complexity is $O(n)$ embedding calls + $O(k)$ LLM verification calls, where $k \ll n^2$ due to the similarity pre-filter. In practice, with $n = 100$ facts and $\tau = 0.7$, typically fewer than 10 pairs pass the pre-filter, making the algorithm efficient even with expensive LLM backends.

4.1.7 Stage 7: Evolution Arc Detection. The evolution arc detector compares `ThemeSnapshot` objects across adjacent time windows to identify trends. For each theme that appears in multiple windows, the system computes the intensity change and classifies the trend as:

Algorithm 1 Contradiction Detection

Require: Memory store M with facts $F = \{f_1, \ldots, f_n\}$
Require: LLM provider L, embedding provider E
Require: Similarity threshold $\tau = 0.7$, max pairs $k = 20$
Ensure: Set of contradiction pairs C

```
C ← ∅
B ← {f ∈ F : f.category ∈ {Belief, Preference} ∧ f.active ∧ |f.words| > 5}
if |B| < 2 then
    return C
end if
e_i ← E.embed(f_i.text) for all f_i ∈ B {O(n) embeddings}
pairs ← 0
for i = 1 to |B| do
    for j = i + 1 to |B| do
        if pairs ≥ k then
            break all
        end if
        s ← sim(e_i, e_j) {Eq. 1}
        if s > τ then
            {Pre-filter: semantically related}
            r ← L.verify_contradiction(f_i, f_j)
            if r.is_contradiction then
                C ← C ∪ {(f_i, f_j)}
                M.mark_contradicted(f_i, f_j)
            end if
            pairs ← pairs +1
        end if
    end for
end for
return C
```

- **Increasing**: Intensity consistently rises across windows
- **Decreasing**: Intensity consistently falls
- **Stable**: Intensity varies by less than 0.1 across windows
- **Oscillating**: Intensity alternates between higher and lower values

Evolution arcs aggregate these per-theme trends into coherent narratives of change. For example, an arc might capture "career anxiety (decreasing) + creative pursuits (increasing) + work-life balance (stable)" as a coherent pattern of professional transition.

4.1.8 Stage 8: Narrative Generation. The final stage synthesizes evolution arcs, contradictions, and insights into human-readable narratives. The LLM receives the detected evolution arcs and extracted beliefs and is prompted to generate second-person prose:

> *"Over the past year, your writing shows a significant shift in how you think about career growth. In early 2022, you frequently expressed a belief that stability was paramount—phrases like 'I value predictability' and 'risk is irresponsible' appeared across your journal entries. By mid-2023, a different narrative emerged: you began writing about 'embracing uncertainty' and 'growth requires discomfort.' This evolution suggests..."*

Narratives include citations to specific source documents and timestamps, allowing the user to verify claims against their original text. The second-person voice ("you") was chosen based on therapeutic writing research suggesting it promotes self-reflection more effectively than first-person narration [46].

4.2 Time Window Strategy

The analysis pipeline supports five granularity levels: daily, weekly, monthly, quarterly, and yearly, with configurable window sizes per analysis run. The `TimeGranularity` enum maps each level to a window size in days:

Granularity	Window (days)	Typical use
Daily	1	Journaling streaks
Weekly	7	Short-term mood tracking
Monthly	30	Default analysis window
Quarterly	90	Seasonal patterns
Yearly	365	Long-term evolution

Additionally, users can define custom `TimeBoundary` markers representing life events ("started new job," "moved to Berlin," "began therapy") with dates and optional colors. These boundaries appear on the timeline visualization and can be used as custom analysis window boundaries, enabling analysis of personal evolution relative to significant life transitions.

4.3 Prompt Engineering

All LLM prompts are managed through a versioned prompt registry. Each prompt template is identified by a name and version number (e.g., `CONTRADICTION_CHECK_V1`), enabling reproducible analysis runs and prompt iteration without code changes.

Templates use a placeholder syntax (`{variable}`) rendered at runtime by a `render_template` function. All templates request JSON-structured output with explicit field specifications, improving parsing reliability across different LLM providers. A post-processing step strips Markdown code fences (`"'json ... "'`) that some providers add around JSON output, ensuring cross-provider compatibility.

5 Query and Retrieval System

5.1 Hybrid Search

MemryLab implements a hybrid search system combining two complementary retrieval methods:

Full-text search. SQLite FTS5 [39] indexes all document chunks with BM25 ranking [36]. BM25 excels at precise keyword matching and handles named entities, dates, and technical terms that embedding-based search may conflate with semantically similar but factually different content.

Vector similarity search. Document chunks are embedded using the configured embedding provider (default: `nomic-embed-text` with 768 dimensions [38]) and stored in the vector store. Query-time search computes the query embedding and retrieves the top-k chunks by cosine similarity (Eq. 1). Vector search captures semantic similarity, finding relevant content even when the user's query uses different vocabulary than the source text.

Reciprocal Rank Fusion. Results from both systems are combined using Reciprocal Rank Fusion (RRF) [37] with parameter $k = 60$:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)} \tag{2}$$

where R is the set of retrieval systems and $\mathrm{rank}_r(d)$ is the rank of document d in system r. The fused ranking benefits from both systems: keyword-heavy queries are well-served by BM25, while conceptual queries benefit from vector search, and queries with both characteristics benefit from the combination.

5.2 RAG Pipeline

The full RAG pipeline follows six steps:

(1) **Query embedding.** The user's query is embedded using the same model that generated chunk embeddings.
(2) **Parallel retrieval.** FTS5 and vector search run in parallel, each retrieving $2k$ candidates (where k is the desired final result count).
(3) **RRF fusion.** Candidates are merged using Eq. 2 and truncated to the top k.
(4) **Context assembly.** Full chunk text is retrieved for the top k results, formatted with timestamps as `[YYYY-MM-DD] chunk_text`.
(5) **Memory augmentation.** The memory store is queried for facts relevant to the user's query. A two-pass strategy first searches the full query phrase, then individual keywords (for words longer than 3 characters) to broaden recall. Up to 10 memory facts are injected as persistent context alongside retrieved chunks.
(6) **LLM generation.** The assembled context (retrieved chunks + memory facts + user query) is sent to the LLM with a response template requesting a grounded answer with source citations.

Graceful degradation. The pipeline is designed to function at reduced quality when components are unavailable. If embedding fails (e.g., the Ollama server is down), vector search is skipped and the system falls back to FTS5 keyword search plus memory recall. If no memory facts are found, the system proceeds with retrieval-only context. This ensures the system always provides some answer rather than failing entirely.

5.3 Conversational Interface

The query interface supports persistent chat history with conversation management. Each conversation is stored with its message history, enabling multi-turn interactions where the user progressively refines their questions about their personal history.

Source citations in RAG responses link back to original documents and timestamps, allowing the user to verify any claim against the source material. Citations include the chunk ID, document ID, a text snippet, the document timestamp, and the RRF score, providing full provenance for every piece of retrieved information.

6 Privacy Architecture

Personal data analysis systems face a fundamental tension: extracting insights requires sophisticated computation (increasingly, LLM inference), but the data is among the most sensitive information a person possesses—journals, therapy reflections, private messages, and unfiltered thoughts.

6.1 Threat Model

We consider four threat categories:

Device theft or loss. An adversary gains physical access to the device and attempts to read the database.

Cloud provider logging. When cloud LLM APIs are used, the provider may log prompts and responses, potentially exposing personal content.

Malicious extensions. Future plugin or extension systems could attempt to exfiltrate data through covert channels.

Data export risks. The system's own export functionality could inadvertently include sensitive information in exported files.

6.2 Mitigations

Encrypted storage. All data resides in a single SQLite database file. The connection layer supports SQLCipher encryption via a `PRAGMA key` that must be issued before any other database operations. When encryption is enabled, the database is unreadable without the passphrase.

Secure credential storage. API keys for cloud LLM providers are stored in the operating system's secure keychain: Windows Credential Manager on Windows, Keychain Services on macOS, and the Secret Service API on Linux. Keys are never written to configuration files, environment variables, or the SQLite database.

PII detection. A regex-based PII detector scans imported content for patterns matching Social Security numbers (`\d{3}-\d{2}-\d{4}`), credit card numbers, email addresses, US and international phone numbers, dates of birth, and IP addresses. Detected PII types are flagged in the import report, allowing users to review and redact sensitive information before analysis.

Zero telemetry. The application contains no analytics, crash reporting, or telemetry of any kind. No network requests are made except to configured LLM API endpoints, and all such requests are transparently logged with the provider name, model, token count, and wall-clock time.

Transparent cloud usage. When a cloud LLM provider is configured, the usage logging decorator records every API call. The user can review a complete log of what data was sent to which provider, enabling informed decisions about the privacy-performance tradeoff of local versus cloud inference.

Local-first architecture. By default, the system operates entirely offline using a local Ollama instance. Cloud providers are opt-in and require explicit API key configuration. The system prominently indicates in the UI whether the current configuration involves any cloud data transmission.

7 Implementation

MemryLab is implemented as a cross-platform desktop application using Tauri 2.0 [40], a framework that combines a Rust backend with a web-based frontend rendered by the platform's native webview. This architecture yields a 4.7 MB installer, roughly 30× smaller than an equivalent Electron application, while maintaining full cross-platform support for Windows, macOS, and Linux.

The Rust backend comprises 120+ source files organized into four layers following the hexagonal architecture:

- **Domain layer:** Models (`Document`, `MemoryFact`, `Entity`, `ThemeSnapshot`, `Insight`, `TimeBoundary`) and port traits (`IDocumentStore`, `IVectorStore`, `ILlmProvider`, `IEmbeddingProvider`, `IMemoryStore`, `IGraphStore`, `ITimelineStore`, `IPageIndex`).
- **Adapter layer:** SQLite implementations of all stores, LLM provider adapters (Ollama, OpenAI-compatible, Claude), and OS keychain integration.

Table 2. Import performance across data sources. Times include parsing, normalization, chunking, deduplication, and database insertion. Embedding generation is excluded (runs asynchronously).

Source	Files	Documents	Time (s)
Obsidian vault (3 years)	1,247	1,247	3.2
Day One journal (5 years)	1,832	1,832	4.1
Twitter archive	1	8,421	6.3
WhatsApp export (2 chats)	2	4,156	2.8
Google Takeout (full)	342	2,891	8.7
Reddit archive	2	1,203	1.4
Mixed re-import (dedup)	3,424	0	1.1

- **Pipeline layer:** Ingestion (source adapters, ZIP handler, dedup, normalizer, chunker, orchestrator) and analysis (eight pipeline stages).
- **Command layer:** 42 Tauri commands that bridge the frontend to the backend, covering import, analysis, query, entity exploration, export, settings, and authentication.

The frontend is built with React and TypeScript, featuring eight primary views: Dashboard, Timeline, Documents, Entities, Insights, Chat, Evolution, and Settings. Data visualizations use D3.js [41] for the timeline view and knowledge graph explorer, with force-directed layouts for entity relationship graphs and stacked area charts for theme evolution over time.

The application is open source under the MIT license[2].

8 Evaluation

We evaluate MemryLab across four dimensions: import performance, analysis quality, system efficiency, and a longitudinal case study.

8.1 Quantitative Analysis

8.1.1 Import Performance. Table 2 shows import throughput across representative data sources. All measurements were taken on a consumer laptop (AMD Ryzen 7 7840U, 32 GB RAM, NVMe SSD) with the SQLite database in WAL mode.

Notably, re-importing the same data after a full import completes in 1.1 seconds with zero new documents, demonstrating the efficiency of SHA-256 deduplication. The system processes approximately 400–500 documents per second for text-based formats, with JSON parsing (Twitter, Telegram) being somewhat slower due to structure extraction overhead.

8.1.2 Analysis Quality. We evaluated entity extraction quality by manually annotating 200 randomly sampled chunks from a mixed personal corpus, then comparing against MemryLab's extraction results. Table 3 reports precision, recall, and F1 by entity type.

Person entities achieve the highest F1 (0.87), likely because personal text frequently mentions people by name in unambiguous contexts. Concept entities have the lowest F1 (0.72), reflecting the inherent subjectivity in deciding what constitutes a meaningful "concept" in personal writing.

[2]https://github.com/tlaad/memrylab

Table 3. Entity extraction quality on 200 manually annotated chunks (using Llama 3.1 8B via Ollama).

Entity Type	Precision	Recall	F1
Person	0.91	0.84	0.87
Place	0.88	0.79	0.83
Organization	0.85	0.72	0.78
Concept	0.76	0.68	0.72
Overall	0.85	0.76	0.80

Table 4. Contradiction detection accuracy on 50 synthetic belief pairs (25 contradictions, 25 non-contradictions).

LLM Backend	Precision	Recall	F1
Llama 3.1 8B (local)	0.80	0.72	0.76
Gemini 1.5 Flash (cloud)	0.88	0.84	0.86
GPT-4o-mini (cloud)	0.92	0.88	0.90

Table 5. Search relevance (NDCG@5) across retrieval methods on 30 personal queries over a 5,000-document corpus.

Method	NDCG@5
FTS5 (BM25 only)	0.61
Vector search only	0.58
Hybrid (RRF, k=60)	0.72
Hybrid + Memory augment.	0.78

8.1.3 Contradiction Detection Accuracy. We constructed a synthetic evaluation set of 50 belief pairs: 25 genuine contradictions (e.g., "I believe routine creates productivity" / "I find that spontaneity leads to my best work") and 25 non-contradictions that are semantically related but compatible (e.g., "I enjoy solitude" / "I need alone time to recharge"). Table 4 reports detection accuracy across three LLM backends.

Results confirm that contradiction detection quality scales with model capability, but even the fully-local Llama 3.1 8B model achieves a reasonable F1 of 0.76. False positives typically involve facts that express different aspects of the same topic (nuanced but not contradictory), while false negatives involve subtle contradictions that require multi-step reasoning.

8.1.4 Search Relevance. We evaluated search quality using 30 personal queries (e.g., "What did I think about remote work in 2022?", "Who did I spend time with last summer?") against a corpus of 5,000 documents spanning three years. A human assessor rated the top-5 results for each query on a 3-point relevance scale (0 = irrelevant, 1 = partially relevant, 2 = highly relevant). Table 5 reports NDCG@5 for each retrieval method.

Hybrid search with RRF outperforms either individual method by a substantial margin. Memory augmentation provides an additional 8% improvement by injecting relevant long-term facts that help the LLM contextualize retrieved chunks. The improvement is most pronounced for queries about beliefs and preferences, where memory facts provide direct answers that complement the evidence from retrieved chunks.

Timeline visualization showing writing volume by platform (stacked area), theme intensities (line overlay), and custom life event markers (vertical lines). The participant's career shift correlates with increased Obsidian note-taking and decreased Twitter activity.

Fig. 2. Timeline visualization from the case study showing writing patterns across platforms over 24 months. Life event markers ("started freelancing," "first client project") correlate with shifts in theme intensity and platform usage.

8.2 Case Study: Two Years of Personal Evolution

To demonstrate the system's analytical capabilities, we conducted a detailed case study with a volunteer (age 32, software engineer) who imported two years of personal data.

Data imported. The participant imported Google Takeout (email metadata, search history, YouTube watch history), three years of Day One journal entries (1,832 entries), an Obsidian vault (847 notes), a Twitter archive (2,100 tweets), and two WhatsApp chat exports. Total: 14,653 documents after deduplication.

Themes extracted. Monthly theme extraction identified 47 unique themes across 24 time windows. The five most persistent themes were: career development (appeared in 22/24 months), health and fitness (20/24), creative writing (18/24), social relationships (17/24), and financial planning (15/24).

Beliefs found. The belief extractor identified 127 distinct beliefs and preferences. Categories broke down as: 43 beliefs, 51 preferences, 18 self-descriptions, and 15 insights.

Contradictions detected. The contradiction detector identified four belief pairs with contradictions. The most striking example:

> **Fact A** (Journal, March 2022): "I value stability and predictability in my career. Taking risks feels irresponsible when you have people depending on you."
> **Fact B** (Twitter, November 2023): "The biggest growth moments in my career came from embracing uncertainty. Playing it safe is the real risk."

The system correctly classified this as a high-severity contradiction and noted the 20-month temporal gap between the statements. When surfaced to the participant, they reported being surprised by the explicit documentation of this shift, noting: "I knew I'd changed my mind about this, but I didn't realize how starkly my old writing contradicts what I believe now."

Evolution narrative. The narrative generator produced a 400-word synthesis describing the participant's career philosophy evolution, noting the transition from a risk-averse to a growth-oriented mindset, correlated with a period of increased writing about entrepreneurship and creativity in their Obsidian notes (Figure 2).

Table 6. LLM API calls and approximate token usage for a full eight-stage analysis run on 14,653 documents.

Stage	Calls	Input (k tok)	Output (k tok)
Theme extraction	24	72	12
Sentiment tracking	20	16	2
Belief extraction	20	40	10
Entity extraction	15	30	6
Insight generation	5	20	5
Contradiction det.	12	10	3
Evolution arcs	6	15	4
Narrative generation	3	12	4
Total	**105**	**215**	**46**

8.3 System Performance

Binary size. The MemryLab installer is 4.7 MB on Windows, compared to approximately 150 MB for a typical Electron application with equivalent functionality. This 30× reduction is attributable to Tauri's use of the platform's native webview rather than bundling Chromium.

Memory usage. At idle, the application consumes approximately 45 MB of RAM. During a full analysis run on 15,000 documents, peak memory usage reaches approximately 180 MB. During import of a large archive (10,000+ files), peak memory is approximately 220 MB. These figures are well within the capacity of any modern desktop computer.

LLM efficiency. Table 6 summarizes the LLM API calls required for a full analysis run on the case study corpus.

The full pipeline requires approximately 105 LLM calls consuming 261K tokens total. At current Ollama local inference rates (approximately 30 tokens/second on the test hardware with Llama 3.1 8B), a full analysis completes in approximately 25 minutes. With cloud APIs (e.g., Gemini Flash at 1M tokens/$0.075), the monetary cost is approximately $0.02 per full analysis run.

8.4 Limitations

LLM quality variance. Analysis quality varies substantially across LLM providers. The 14-point F1 gap between Llama 3.1 8B and GPT-4o-mini in contradiction detection (Table 4) illustrates this tradeoff between privacy (local inference) and quality (cloud models). Users must balance these concerns based on their individual privacy requirements.

Embedding quality bottleneck. Contradiction detection pre-filtering depends on embedding quality. The similarity threshold $\tau = 0.7$ was tuned for `nomic-embed-text`; different embedding models may require different thresholds. Poor embeddings can cause both false positives (unrelated facts with spuriously high similarity) and false negatives (genuine contradictions below the threshold).

No multimedia analysis. The current system processes only text. Images, audio recordings, and video content in data exports are ignored. This is a significant limitation given that platforms like Instagram and TikTok are primarily visual or audiovisual.

Single-user design. MemryLab is designed for individual use. There is no multi-user support, collaborative analysis, or shared knowledge base functionality. While this simplifies the privacy model, it precludes use cases such as couples reflecting on shared communication patterns.

Evaluation scope. The quantitative evaluations presented in this paper rely on a single participant's data and synthetic belief pairs. A larger-scale user study with diverse participants, data volumes, and cultural contexts is needed

to validate the generalizability of the system's analytical capabilities and the practical utility of the detected belief evolution patterns.

Platform export stability. The 33 source adapters depend on the current export formats of their respective platforms. These formats are undocumented and subject to change without notice, which means adapters may require ongoing maintenance to remain functional.

8.5 Ethics Statement

This work involves the analysis of deeply personal data—journals, private messages, social media posts, and other intimate digital traces. We identify three primary ethical considerations.

First, the system processes data that may include communications from third parties (e.g., messages in group chats) without those parties' explicit consent. While users have a legal right to download and process their own data exports under GDPR Article 20 and similar regulations, analyzing others' words raises ethical questions that users should consider. The system does not distinguish between the user's own text and others' text within conversation exports.

Second, automated belief tracking and contradiction detection could be misused for self-surveillance that reinforces rumination or anxiety rather than promoting healthy self-reflection. The system is designed as a reflection aid, not a diagnostic or therapeutic tool, and should not be used as a substitute for professional mental health support.

Third, while the local-first architecture provides strong privacy guarantees by default, users who opt into cloud LLM providers should understand that their personal text is transmitted to third-party servers for inference. The system mitigates this through transparent usage logging, but the ultimate responsibility for this privacy tradeoff rests with the user.

8.6 Open Science

MemryLab is released as open-source software under the MIT license. The complete source code, including all 33 source adapters, the eight-stage analysis pipeline, and the hybrid RAG system, is available at https://github.com/tlaad/memrylab. We encourage the research community to build upon this work, contribute additional source adapters, and conduct independent evaluations with diverse user populations.

9 Discussion

Implications for self-reflection and therapy. The ability to surface belief contradictions and evolution patterns has potential applications in therapeutic contexts. Cognitive behavioral therapy (CBT) often involves identifying distorted thinking patterns [42]; a system that automatically surfaces contradictions in a patient's written output could accelerate this process. However, we caution that automated belief analysis is not a substitute for clinical judgment, and the system should be positioned as a reflection aid rather than a diagnostic tool.

Quantified self community. The quantified self movement [43] has focused primarily on physiological data (steps, heart rate, sleep). MemryLab extends this paradigm to the cognitive domain, enabling "quantified beliefs" and "quantified values." The theme intensity scores and sentiment trends provide numerical metrics for self-tracking, while the narrative synthesis provides qualitative context.

Ethical considerations. Personal belief tracking raises ethical concerns. Self-surveillance can become pathological, reinforcing rumination rather than promoting growth [44]. There is also a risk of confirmation bias: users may selectively attend to evolution narratives that confirm their desired self-image while dismissing contradicting evidence. The system

attempts to mitigate this by presenting contradictions prominently and including raw source citations that users can verify independently.

Additionally, if data exports include messages from other people (e.g., group chats), the system processes those messages without the consent of the other participants. While this is legally permissible (the user downloaded their own data export), it raises ethical questions about analyzing others' words without their knowledge.

Future work. Several extensions are planned. A plugin system would allow third-party developers to add new source adapters without modifying the core codebase. Multi-device synchronization via CRDTs (Conflict-free Replicated Data Types) [45] would enable the local-first architecture to span multiple devices without centralized servers. A mobile companion application would enable journaling on-the-go with automatic synchronization to the desktop analysis engine. Finally, multimedia analysis—using vision-language models to extract themes from images and speech-to-text for audio—would address the current limitation of text-only processing.

10 Conclusion

We have presented MemryLab, a privacy-preserving system for tracking personal belief evolution through multi-source digital footprint analysis. The system unifies data from 30+ platforms through a confidence-scored adapter architecture, applies an eight-stage LLM-powered analysis pipeline to extract themes, sentiments, beliefs, entities, contradictions, and evolution narratives, and provides a hybrid RAG-based conversational interface for exploring personal history.

The contradiction detection algorithm—combining embedding-based semantic pre-filtering with LLM verification—achieves F1 scores between 0.76 (local inference) and 0.90 (cloud models) on synthetic belief pairs, while maintaining sub-quadratic complexity that makes it practical for large personal fact stores.

The system's local-first architecture ensures that personal data never leaves the user's device unless they explicitly opt into cloud LLM usage, with transparent logging of all external API calls. The Tauri 2.0 implementation delivers this functionality in a 4.7 MB binary—30× smaller than equivalent Electron applications.

As people generate increasingly voluminous digital footprints across an expanding array of platforms, the gap between data generation and self-understanding continues to widen. MemryLab represents a step toward closing that gap, transforming scattered digital traces into coherent narratives of personal evolution. The system is available as open-source software under the MIT license.

References

[1] Vannevar Bush. As We May Think. *The Atlantic Monthly*, 176(1):101–108, July 1945.
[2] Jim Gemmell, Gordon Bell, Roger Lueder, Steven Drucker, and Curtis Wong. MyLifeBits: Fulfilling the Memex Vision. In *Proceedings of ACM Multimedia*, pages 235–238, 2002.
[3] Gordon Bell and Jim Gemmell. *Total Recall: How the E-Memory Revolution Will Change Everything.* Dutton, 2009.
[4] Microsoft. Retrace Your Steps with Recall. Microsoft Windows Blog, May 2024. https://blogs.windows.com/windowsexperience/2024/05/20/recall/.
[5] William Jones. *Keeping Found Things Found: The Study and Practice of Personal Information Management.* Morgan Kaufmann, 2007.
[6] Mark Lansdale. The Psychology of Personal Information Management. *Applied Ergonomics*, 19(1):55–66, 1988.
[7] Steve Whittaker. Personal Information Management: From Information Consumption to Curation. *Annual Review of Information Science and Technology*, 45(1):1–62, 2011.
[8] Richard Boardman and M. Angela Sasse. "Stuff Goes into the Computer and Doesn't Come Out": A Cross-tool Study of Personal Information Management. In *Proceedings of CHI*, pages 583–590. ACM, 2004.
[9] Krisztian Balog and Tom Kenter. Personal Knowledge Graphs: A Research Agenda. In *Proceedings of ICTIR*, pages 217–220. ACM, 2019.
[10] Mayank Kejriwal. *Knowledge Graphs: Fundamentals, Techniques, and Applications.* MIT Press, 2022.
[11] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. A Survey on Deep Learning for Named Entity Recognition. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):50–70, 2020.

[12] Vikas Yadav and Steven Bethard. A Survey on Recent Advances in Named Entity Recognition from Deep Learning Models. In *Proceedings of COLING*, pages 2145–2158, 2018.
[13] Cunchao Jiang, Jianghong Ma, and Xiao Qin. A Survey on Temporal Knowledge Graph: Representation, Completion, and Applications. *arXiv preprint arXiv:2403.04782*, 2024.
[14] Julien Leblay and Melisachew Wudage Chekol. Deriving Validity Time in Knowledge Graphs. In *Companion Proceedings of The Web Conference*, pages 1771–1776, 2018.
[15] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*, 2025.
[16] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. In *Proceedings of the ACM Web Conference 2025*, 2025.
[17] Jiayan Nan et al. Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science. *arXiv preprint arXiv:2508.03341*, 2025.
[18] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. *arXiv preprint arXiv:2512.12818*, 2025.
[19] Shreyas Rajesh, Pavan Holur, Chenda Duan, David Chong, and Vwani Roychowdhury. Beyond Fact Retrieval: Episodic Memory for RAG with Generative Semantic Workspaces. *arXiv preprint arXiv:2511.07587*, 2025.
[20] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. Knowledge Conflicts for LLMs: A Survey. In *Proceedings of EMNLP*, 2024.
[21] Michal Kosinski, David Stillwell, and Thore Graepel. Private Traits and Attributes Are Predictable from Digital Records of Human Behavior. *Proceedings of the National Academy of Sciences*, 110(15):5802–5805, 2013.
[22] Wu Youyou, Michal Kosinski, and David Stillwell. Computer-Based Personality Judgments Are More Accurate Than Those Made by Humans. *Proceedings of the National Academy of Sciences*, 112(4):1036–1040, 2015.
[23] Mike Thelwall, Kevan Buckley, and Georgios Paltoglou. Sentiment Strength Detection for the Social Web. *Journal of the American Society for Information Science and Technology*, 63(1):163–173, 2012.
[24] Scott A. Golder and Michael W. Macy. Diurnal and Seasonal Mood Vary with Work, Sleep, and Daylength Across Diverse Cultures. *Science*, 333(6051):1878–1881, 2011.
[25] Jiwei Li, Alan Ritter, Claire Cardie, and Eduard Hovy. Major Life Event Extraction from Twitter Based on Congratulations/Condolences Speech Acts. In *Proceedings of EMNLP*, pages 1997–2007, 2014.
[26] Munmun De Choudhury, Michael Gamon, Scott Counts, and Eric Horvitz. Predicting Depression via Social Media. In *Proceedings of ICWSM*, 2013.
[27] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of Onward!*, pages 154–178. ACM, 2019.
[28] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of AISTATS*, pages 1273–1282, 2017.
[29] Georgi Gerganov. llama.cpp: Inference of LLaMA Model in Pure C/C++. GitHub repository, 2023. https://github.com/ggerganov/llama.cpp.
[30] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized Language Models. In *Proceedings of NeurIPS*, 2023.
[31] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language Models Are Few-Shot Learners. In *Proceedings of NeurIPS*, 2020.
[32] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.
[33] Alistair Cockburn. Hexagonal Architecture. *alistair.cockburn.us*, 2005. https://alistair.cockburn.us/hexagonal-architecture/.
[34] European Parliament and Council. Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, L119:1–88, 2016.
[35] Simon Kemp. Digital 2024: Global Overview Report. DataReportal, January 2024. https://datareportal.com/reports/digital-2024-global-overview-report.
[36] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
[37] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of SIGIR*, pages 758–759. ACM, 2009.
[38] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv preprint arXiv:2402.01613*, 2024.
[39] D. Richard Hipp. SQLite FTS5 Extension. SQLite Documentation, 2023. https://www.sqlite.org/fts5.html.
[40] Daniel Thompson-Yvetot, Lucas Nogueira, et al. Tauri: Build Smaller, Faster, and More Secure Desktop and Mobile Applications. https://v2.tauri.app, 2024.
[41] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. D^3: Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.
[42] Aaron T. Beck. *Cognitive Therapy and the Emotional Disorders*. Penguin Books, 1979.
[43] Gary Wolf. The Data-Driven Life. *The New York Times Magazine*, April 2010.
[44] Minna Ruckenstein. Visualized and Interacted Life: Personal Analytics and Engagements with Data Doubles. *Societies*, 4(1):68–84, 2014.

[45] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. In *Proceedings of SSS*, pages 386–400. Springer, 2011.
[46] Timothy D. Wilson. *Redirect: The Surprising New Science of Psychological Change.* Little, Brown, 2011.